

Quick Sort

COP 3502C – Computer Science I

Spring 2026

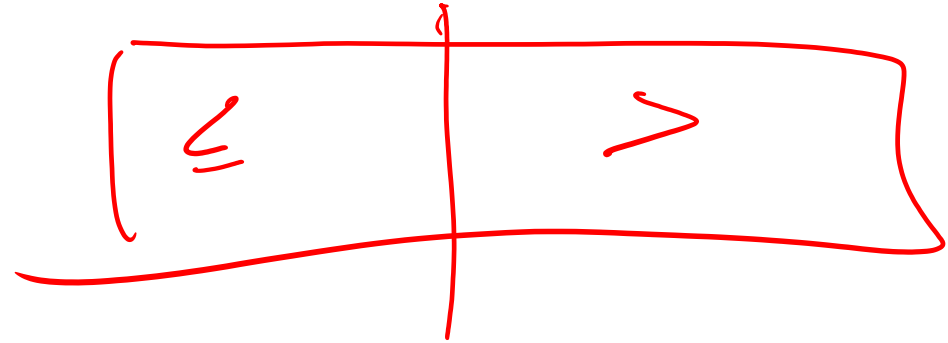
Yancy Vance Paredes, PhD

Recall

pivot

- We talked about the divide and conquer strategy
- It involves dividing a problem into sub-problems
- We solve the smaller problem then combine the results
- Now, we explore another divide and conquer sorting algorithm

Quick Sort /1



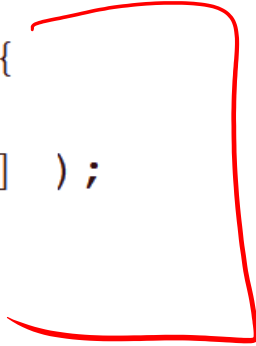
- Given an array, find a **pivot** that will **divide** it into left and right
- One strategy is to use the *last element* as the pivot
- Elements are **partitioned** based on their values *w.r.t.* the pivot
- We are implicitly searching for the pivot's final location (middle)

Quick Sort /2

- Instead of shifting elements similar to insertion sort, we simply just perform a **swap**
- We **recursively solve** the left and the right by applying quick sort
- ✱ • *No need to combine the results*

```
void quick_sort(int *arr, int start, int end) {  
    if( start >= end )  
        return;  
  
    int p_idx = partition(arr, start, end); → Dived  
  
    quick_sort(arr, start, p_idx-1); left  
    quick_sort(arr, p_idx+1, end); right  
}
```

```
int partition(int *arr, int start, int end) {  
    int pivot = arr[end];  
    int d_idx = start;  
  
    for(int i = start; i < end; i++) {  
        if( arr[i] <= pivot ) {  
            swap( &arr[i], &arr[d_idx] );  
            d_idx++;  
        }  
    }  
  
    swap( &arr[end], &arr[d_idx] );  
  
    return d_idx;  
}
```



8	3	1	7	0	10	2
---	---	---	---	---	----	---



8	3	1	7	0	10	2
---	---	---	---	---	----	---

p_{idx}

8	3	1	7	0	10	2
---	---	---	---	---	----	---

d_{idx}

p_{idx}



8	3	1	7	0	10	2
---	---	---	---	---	----	---

d_{idx}

p_{idx}

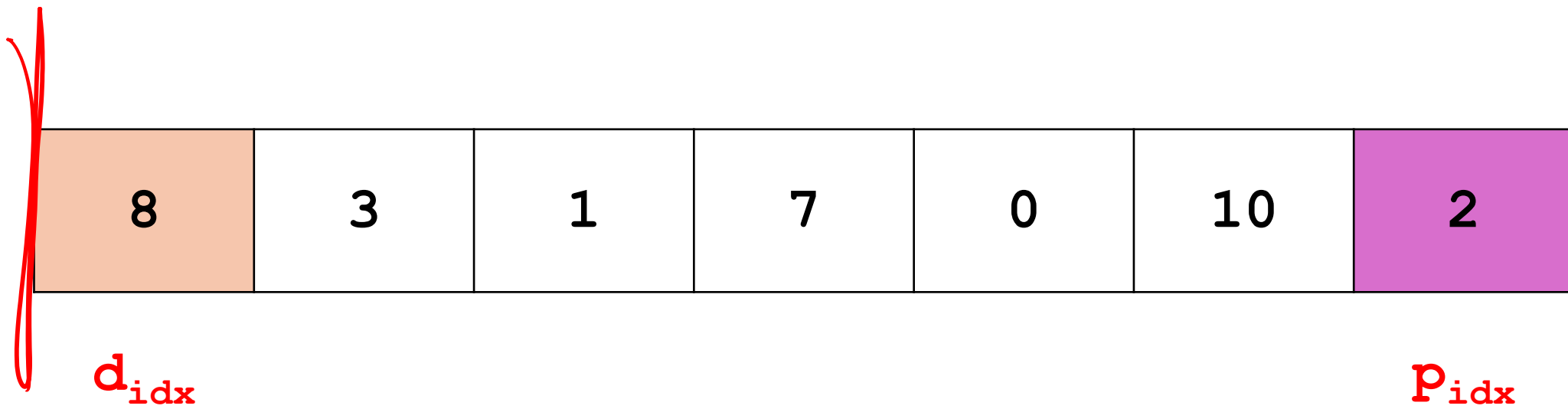
i

8	3	1	7	0	10	2
---	---	---	---	---	----	---

d_{idx}

p_{idx}

i

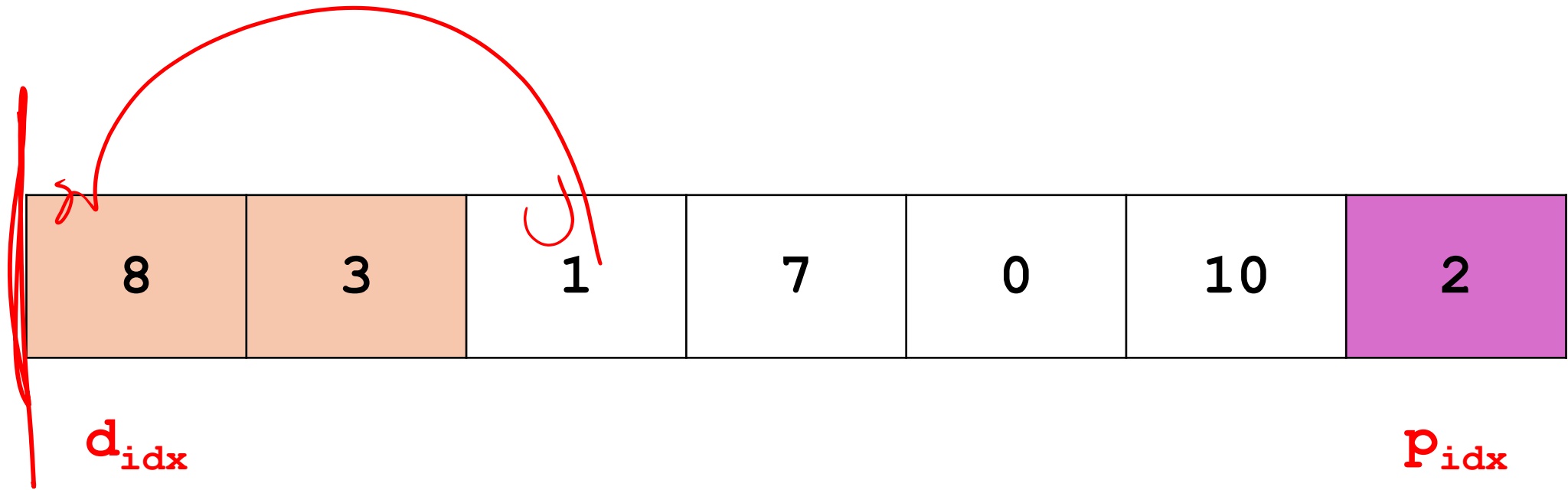


8	3	1	7	0	10	2
---	---	---	---	---	----	---

d_{idx}

p_{idx}

i



need space

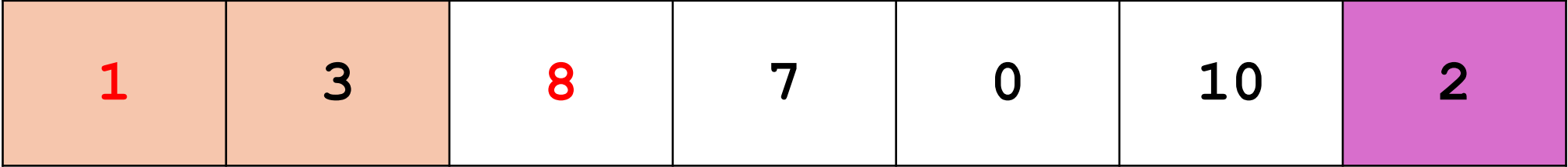
8	3	1	7	0	10	2
---	---	---	---	---	----	---

d_{idx}

p_{idx}

i

swap



1	3	8	7	0	10	2
---	---	---	---	---	----	---

d_{idx}

p_{idx}

i

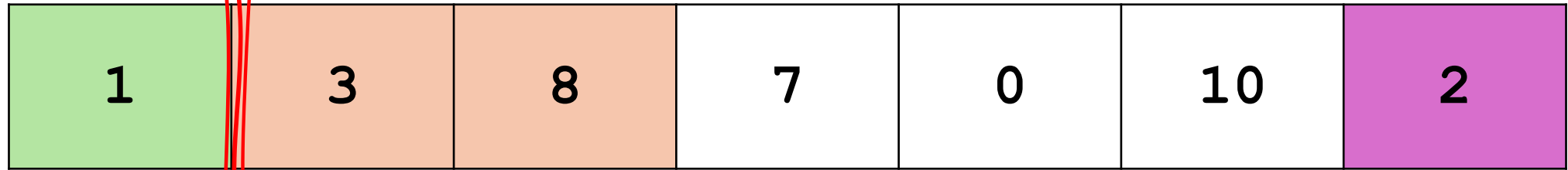
1	3	8	7	0	10	2
---	---	---	---	---	----	---

d_{idx}

p_{idx}

i

move divider



d_{idx}

p_{idx}

i

1	3	8	7	0	10	2
---	---	---	---	---	----	---

d_{idx}

p_{idx}

i

1	3	8	7	0	10	2
---	---	---	---	---	----	---

d_{idx}

p_{idx}

i

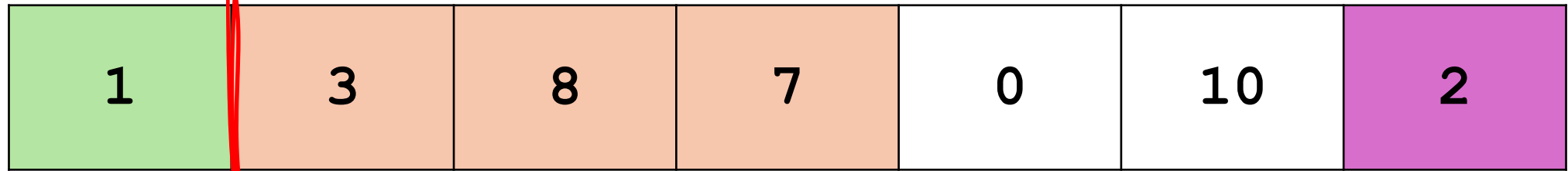
1	3	8	7	0	10	2
---	---	---	---	---	----	---

d_{idx}

p_{idx}

i

need space



d_{idx}

p_{idx}

i

swap

1	0	8	7	3	10	2
---	---	---	---	---	----	---

d_{idx}

p_{idx}

i

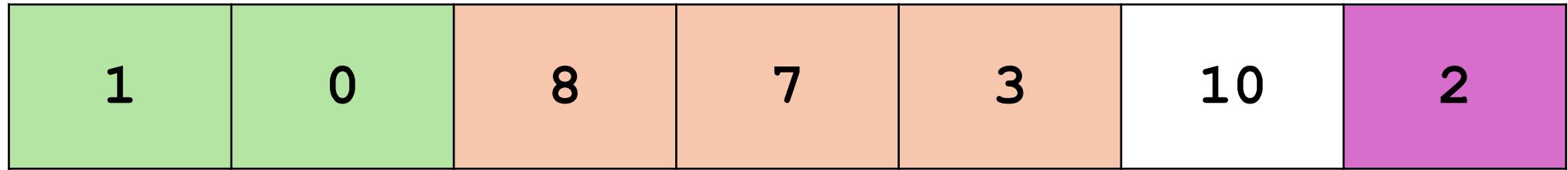
1	0	8	7	3	10	2
---	---	---	---	---	----	---

d_{idx}

p_{idx}

i

move divider



d_{idx}

p_{idx}

i

1	0	8	7	3	10	2
---	---	---	---	---	----	---

d_{idx}

p_{idx}

i

1	0	8	7	3	10	2
---	---	---	---	---	----	---

d_{idx}

p_{idx}

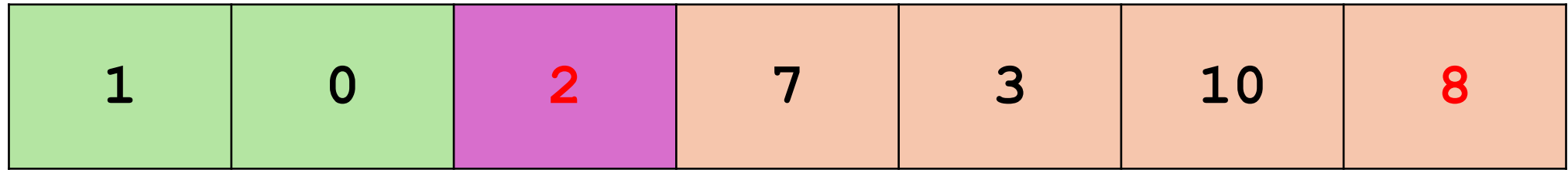
i

1	0	8	7	3	10	2
---	---	---	---	---	----	---

d_{idx}

p_{idx}

swap pivot to final location



d_{idx}

p_{idx}

1	0	2	7	3	10	8
---	---	---	---	---	----	---

d_{idx}

p_{idx}

1	0	2	7	3	10	8
---	---	---	---	---	----	---

1	0	2	7	3	10	8
---	---	---	---	---	----	---

1	0	2	7	3	10	8
---	---	---	---	---	----	---

p_{idx}

1	0	2	7	3	10	8
---	---	---	---	---	----	---

d_{idx}

p_{idx}

1	0	2	7	3	10	8
---	---	---	---	---	----	---

d_{idx}

p_{idx}

i

1	0	2	7	3	10	8
---	---	---	---	---	----	---

d_{idx}

p_{idx}

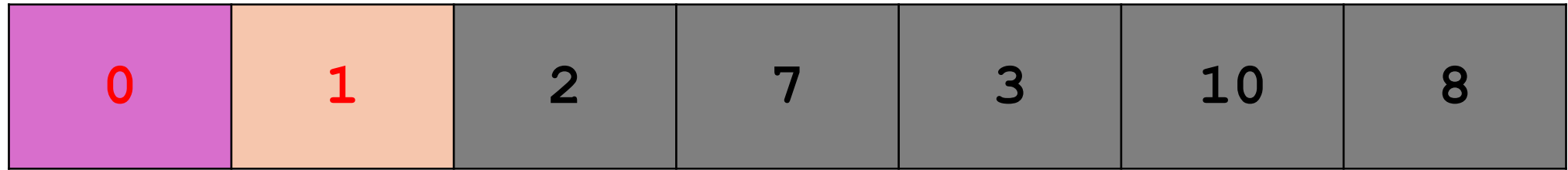
i

1	0	2	7	3	10	8
---	---	---	---	---	----	---

d_{idx}

p_{idx}

swap pivot to final location



d_{idx}

p_{idx}

0	1	2	7	3	10	8
---	---	---	---	---	----	---

d_{idx}

p_{idx}

0	1	2	7	3	10	8
---	---	---	---	---	----	---

0	1	2	7	3	10	8
---	---	---	---	---	----	---

0	1	2	7	3	10	8
---	---	---	---	---	----	---

0	1	2	7	3	10	8
---	---	---	---	---	----	---

0	1	2	7	3	10	8
---	---	---	---	---	----	---

0	1	2	7	3	10	8
---	---	---	---	---	----	---

p_{idx}

0	1	2	7	3	10	8
---	---	---	---	---	----	---

d_{idx}

p_{idx}

0	1	2	7	3	10	8
---	---	---	---	---	----	---

d_{idx}

p_{idx}

i

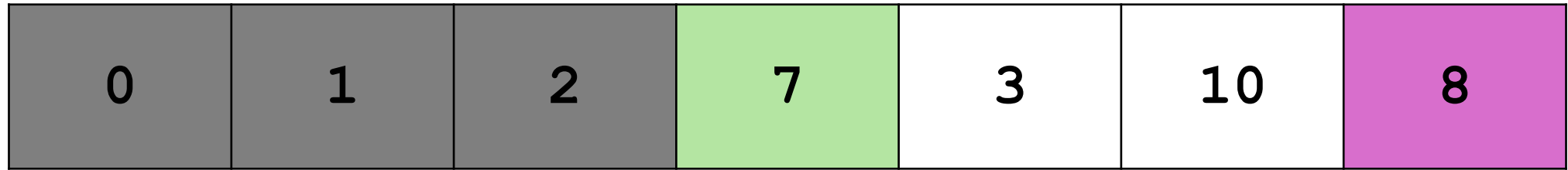
0	1	2	7	3	10	8
---	---	---	---	---	----	---

d_{idx}

p_{idx}

i

move divider



d_{idx}

p_{idx}

i

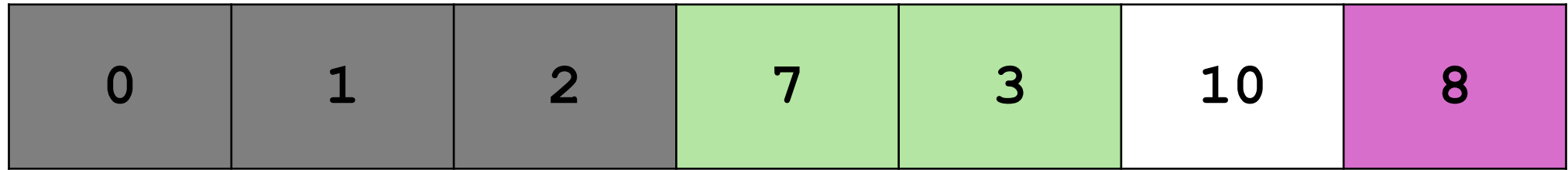
0	1	2	7	3	10	8
---	---	---	---	---	----	---

d_{idx}

p_{idx}

i

move divider



d_{idx}

p_{idx}

i

0	1	2	7	3	10	8
---	---	---	---	---	----	---

d_{idx}

p_{idx}

i

0	1	2	7	3	10	8
---	---	---	---	---	----	---

d_{idx}

p_{idx}

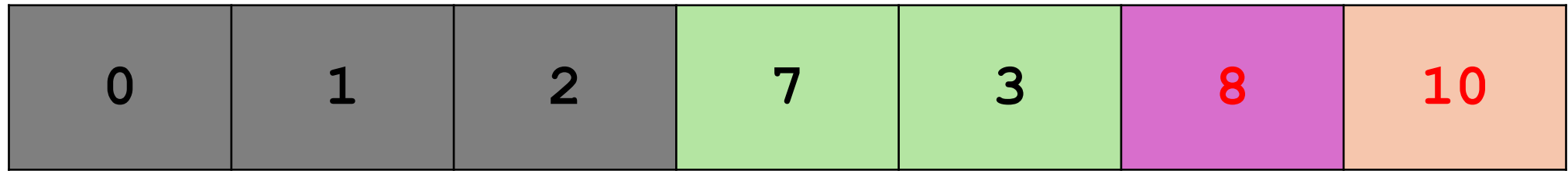
i

0	1	2	7	3	10	8
---	---	---	---	---	----	---

d_{idx}

p_{idx}

swap pivot to final location



d_{idx}

p_{idx}

0	1	2	7	3	8	10
---	---	---	---	---	---	----

0	1	2	7	3	8	10
---	---	---	---	---	---	----

0	1	2	7	3	8	10
---	---	---	---	---	---	----

p_{idx}

0	1	2	7	3	8	10
---	---	---	---	---	---	----

d_{idx}

p_{idx}

0	1	2	7	3	8	10
---	---	---	---	---	---	----

d_{idx}

p_{idx}

i

0	1	2	7	3	8	10
---	---	---	---	---	---	----

d_{idx}

p_{idx}

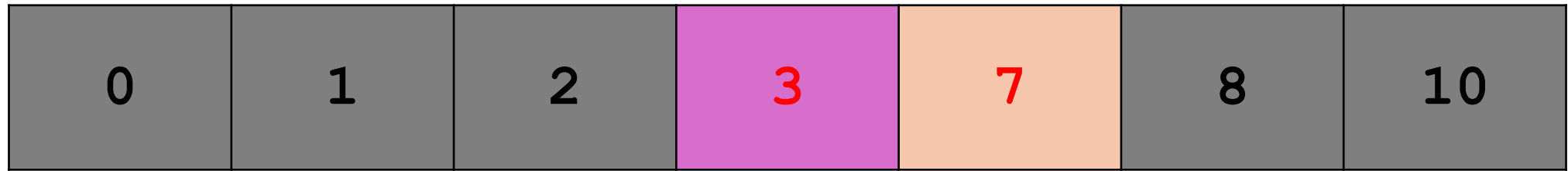
i

0	1	2	7	3	8	10
---	---	---	---	---	---	----

d_{idx}

p_{idx}

swap pivot to final location



d_{idx}

p_{idx}

0	1	2	3	7	8	10
---	---	---	---	---	---	----

0	1	2	3	7	8	10
---	---	---	---	---	---	----

0	1	2	3	7	8	10
---	---	---	---	---	---	----

0	1	2	3	7	8	10
---	---	---	---	---	---	----

0	1	2	3	7	8	10
---	---	---	---	---	---	----

0	1	2	3	7	8	10
---	---	---	---	---	---	----

0	1	2	3	7	8	10
---	---	---	---	---	---	----

0	1	2	3	7	8	10
---	---	---	---	---	---	----

Now: quicksort(0, 6)

P: (2)

L: quicksort(0, 1)

Now: quicksort(0, 1)

P: (0)

L: quicksort(0, -1)

Now: quicksort(0, -1)

Base: Invalid!

R: quicksort(1, 1)

Now: quicksort(1, 1)

Base: P: (1)

R: quicksort(3, 6)

Now: quicksort(3, 6)

P: (5)

L: quicksort(3, 4)

Now: quicksort(3, 4)

P: (3)

L: quicksort(3, 2)

Now: quicksort(3, 2)

Base: Invalid!

R: quicksort(4, 4)

Now: quicksort(4, 4)

Base: P: (4)

R: quicksort(6, 6)

Now: quicksort(6, 6)

Base: P: (6)

Notes /1

- The partition splits the input into two sub-arrays: left and right
- Unlike merge sort, sizes of the sub-arrays may significantly differ
- The size of the sub-arrays are impacted by the value of the pivot

Notes /2

- Interestingly, after performing a quick sort on the sub-arrays, the overall array becomes sorted
- Therefore, there is no need to combine these explicitly

Discussion

- How the **pivot** is determined impacts the performance of the sort
- What is the running time of quick sort?

Check Your Understanding

Sorting Algorithm	Is Stable?	Worst-Case	Average Case	Best-Case
Selection Sort			$O(n^2)$	
Insertion Sort			$O(n^2)$	
Bubble Sort			$O(n^2)$	
Bubble Sort (Enhanced)			$O(n^2)$	
Merge Sort			$O(n \lg n)$	
Quick Sort			$O(n \lg n)$	

Questions?